Twentieth Implementation Iteration: Allocation Strategy Factory

with Singleton-Based Strategy Selection

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this twentieth implementation iteration is to improve the architectural flexibility of the prototype by introducing a factory for allocation strategy selection.

In previous iterations, the system already supported allocation strategies through `IAllocationStrategy` and `GreedyAllocationStrategy`. The allocation service delegated processing behavior to a strategy, and the current greedy algorithm was already explicit. However, the service still depended on the concrete strategy more directly than necessary.

In this iteration, an `AllocationStrategyFactory` is introduced and implemented as a Singleton. The purpose of this factory is to provide allocation strategies through the base interface rather than requiring client code to manually depend on a concrete strategy class. This improves extensibility and prepares the system for future strategy additions while preserving the current greedy behavior.

2. Changes from Iteration 19

Compared with the nineteenth iteration, the following changes were introduced:

- Added a new `AllocationStrategyFactory` class
- Implemented the factory as a Singleton
- Moved strategy selection responsibility into the factory
- Updated `AllocationService` so that it requests its strategy through the factory
- Preserved the use of the `IAllocationStrategy` interface as the abstraction point
- Preserved the current greedy request-processing behavior

- Added safe fallback behavior so unknown strategy names still resolve to the greedy strategy

This iteration does not add a new allocation algorithm. It only improves the way strategies are selected and supplied to the rest of the system.

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files through the `DataController`. After loading is completed, the main program creates an `AllocationService`. Instead of constructing a concrete allocation strategy directly, the service now obtains its strategy through the `AllocationStrategyFactory`.

The factory returns the selected strategy through the `IAllocationStrategy` interface. In the current implementation, the selected strategy is still the greedy strategy. The allocation service then passes the loaded request collection to that strategy, which processes requests in CSV order using the existing rule engine facade.

If a request passes all implemented rules, it is approved and allocations are created immediately. If a request fails, it is rejected and the rejection reason is preserved. Invalid requests and lifecycle history behavior remain unchanged. At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`
- request lifecycle history together with each exported request result

4. Modules and Their Tasks

A. Allocation Strategy Interface Module

- Continue defining the common interface for allocation algorithms through `IAllocationStrategy`
- Allow client code to depend on the strategy abstraction rather than a specific algorithm class

B. Greedy Allocation Module

- Continue implementing the current greedy allocation behavior
- Process requests in loaded order

- Approve and allocate immediately when rules pass
- Reject immediately when rules fail

C. Allocation Strategy Factory Module

- Centralize strategy selection in one dedicated component
- Return strategies through the `IAllocationStrategy` interface
- Provide safe fallback to the greedy strategy for unknown names
- Ensure a single shared factory object through the Singleton pattern

D. Allocation Service Module

- Continue coordinating allocation processing
- Continue owning the allocation list
- Obtain the strategy from the factory instead of directly depending on a concrete strategy object

E. Data and Export Module

- Continue loading users, spaces, and requests from CSV files
- Continue exporting allocations and processed request results
- Preserve request lifecycle history visibility in exported results

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
16	Externalized requests	Class + Lab + Meet	DataController + CSV request loading
17	Same + history log	Class + Lab + Meet	Request lifecycle history logging + export
18	Same + strategy-based processing	Class + Lab + Meet	Allocation Strategy Pattern + Greedy strategy
19	Same + explicit greedy batch flow	Class + Lab + Meet	Greedy batch processing moved into strategy
20	Same + factory-based strategy selection	Class + Lab + Meet	Singleton strategy factory + interface-based selection

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the new `AllocationStrategyFactory`, its Singleton role, and the updated relationship between `AllocationService` and the strategy selection mechanism.

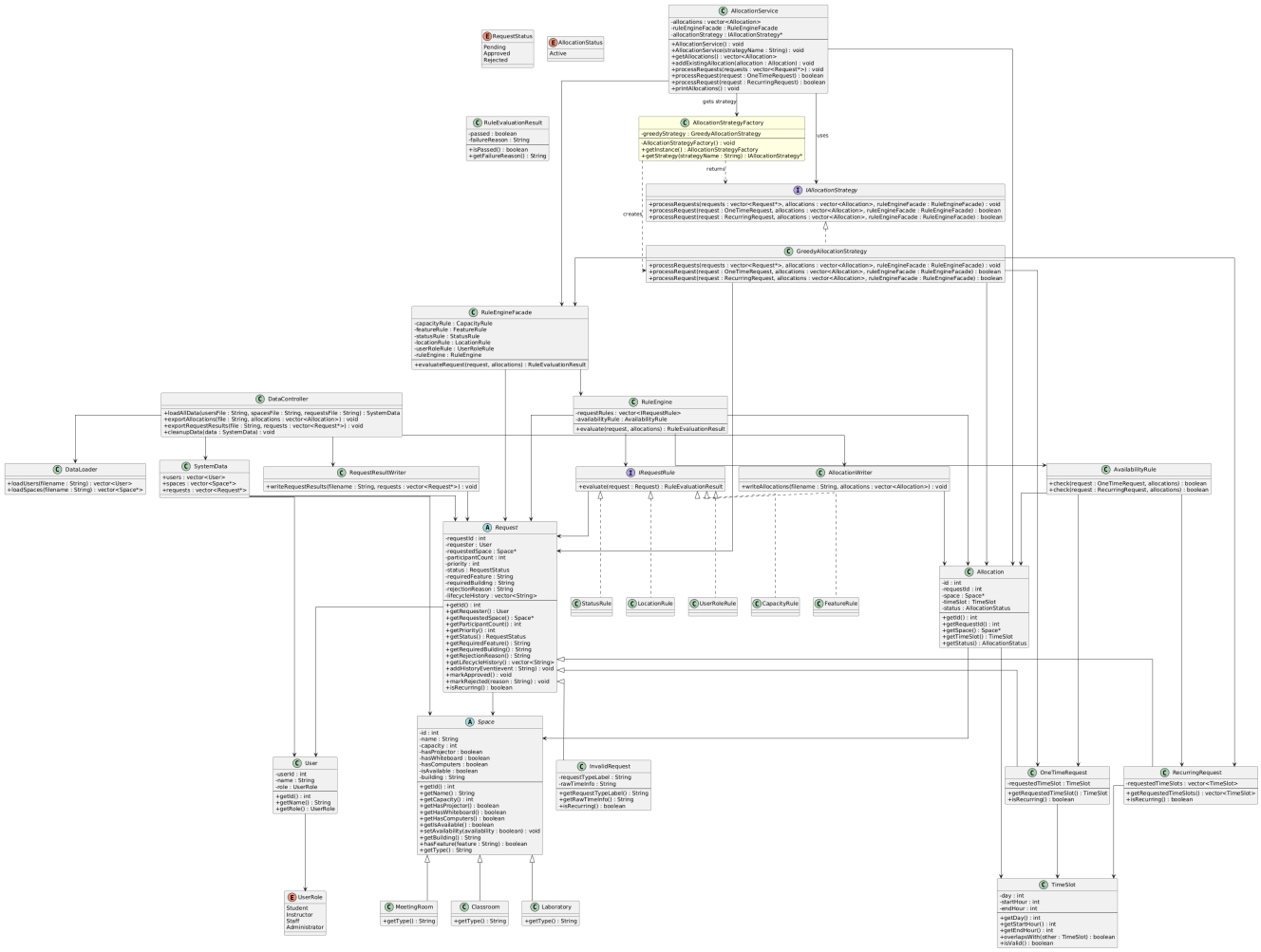

Figure 1: Updated Class Diagram with Allocation Strategy Factory

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The processing behavior remains greedy, but the selection of the strategy is now performed through the factory before request processing begins.

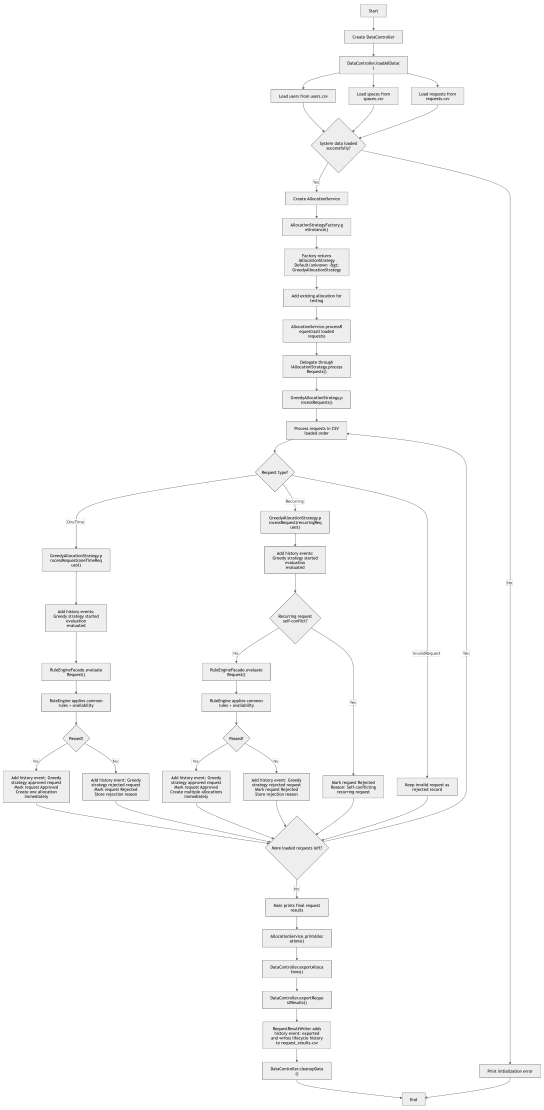

Figure 2: Workflow with Factory-Based Strategy Selection

8. Prototype Implementation

The C++ prototype was extended by introducing `AllocationStrategyFactory` as a Singleton. This factory is responsible for returning allocation strategies through the `IAllocationStrategy` interface. In the current implementation, the factory supports the greedy strategy and safely falls back to greedy if an unknown strategy name is requested.

The `AllocationService` was updated so that it no longer needs to create or store a

concrete greedy strategy directly. Instead, it requests the strategy through the factory and continues to use the interface. This reduces direct coupling between the service and the concrete strategy implementation.

The current greedy algorithm itself was not changed in this iteration. The system still processes requests in loaded order, uses the existing rule engine and facade for validation, creates allocations immediately after approval, and preserves request lifecycle history and CSV exports. The purpose of this iteration is therefore not to add a new optimization algorithm, but to improve the architecture so that later strategies such as `PriorityAllocationStrategy`, `BestFitAllocationStrategy`, or `PriorityBestFitStrategy` can be added with minimal changes to client code.

9. Summary

This twentieth implementation iteration improves the architecture of the prototype by introducing a Singleton-based strategy factory for allocation strategy selection. The system now obtains strategies through the `IAllocationStrategy` interface rather than depending directly on a concrete strategy class.

This iteration preserves the current greedy allocation behavior and does not add new optimization logic. However, it provides a cleaner and more extensible foundation for future iterations that introduce additional allocation strategies.